INFORMATIONSBLATT

IB01 – Eigene Funktionen schreiben

Dynamische Webseiten mit PHP · LS07 | 2BKI2 · Berufskolleg Technische Assistenz · Schuljahr 2026/2027

🎯 Darum geht es

Sie schreiben eigene Funktionen mit Parametern und Rückgabewert, unterscheiden `return` von `echo`, kennen den Gültigkeitsbereich von Variablen und nutzen wichtige eingebaute Funktionen.

1 | Was ist eine Funktion?

Funktionen kennen Sie schon: `date()`, `number_format()`, `count()`. Sie bekommen Werte in den Klammern (**Parameter**), rechnen etwas aus und liefern ein Ergebnis zurück (**Rückgabewert**). Mit `function` schreiben Sie eigene.

Definieren und aufrufen — beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Funktion</title></head>
<body>

<?php
  // Definition: einmal schreiben
  function begruessung($name) {
    return "Willkommen im Eiscafé, $name!";
  }

  // Aufruf: beliebig oft benutzen
  echo "<p>" . begruessung("Lena") . "</p>";
  echo "<p>" . begruessung("Herr Kaya") . "</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

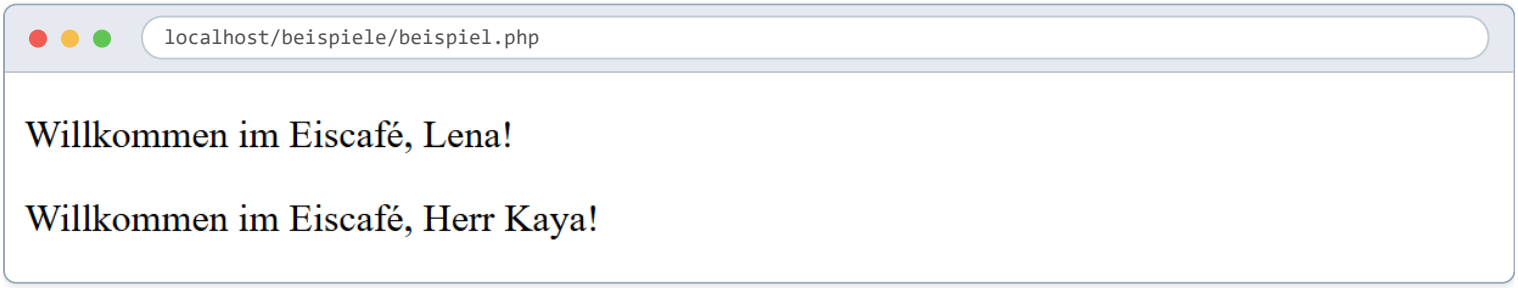


Abbildung 1: Eine Definition, zwei Aufrufe mit unterschiedlichen Werten.

Teil	Bedeutung
`function`	Schlüsselwort: hier beginnt eine Funktionsdefinition
`begruessung`	Name – Regeln wie bei Variablen, aber ohne `$`
`($name)`	Parameter: eine Variable, die beim Aufruf den übergebenen Wert bekommt
`return …;`	gibt das Ergebnis an die Aufrufstelle zurück und beendet die Funktion

Die Funktion euro() beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>euro()</title></head>
<body>

<?php
  function euro($betrag) {
    return number_format($betrag, 2, ",", ".") . " €";
  }

  $summe = 3 * 1.70;
  echo "<p>Drei Kugeln: " . euro($summe) . "</p>";
  echo "<p>Ein Becher: " . euro(7.3) . "</p>";
  echo "<p>Zusammen: " . euro($summe + 7.3) . "</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS



Abbildung 2: Einmal geschrieben, überall benutzt: euro() erledigt das Formatieren.

2 | return oder echo?

Eine Funktion mit `echo` gibt sofort etwas aus – mit dem Ergebnis kann man nicht weiterrechnen. Eine Funktion mit `return` liefert einen **Wert**, über dessen Verwendung die Aufrufstelle entscheidet.

Der Unterschied beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>return oder echo</title></head>
<body>

<?php
  function mit_echo($x)   { echo $x * 2; }
  function mit_return($x) { return $x * 2; }

  $a = mit_echo(5);        // gibt sofort 10 aus – $a bleibt leer
  $b = mit_return(5);      // gibt nichts aus – $b ist 10

  echo "<p>a = '$a', b = '$b'</p>";
  echo "<p>b + 1 = " . ($b + 1) . "</p>";
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS

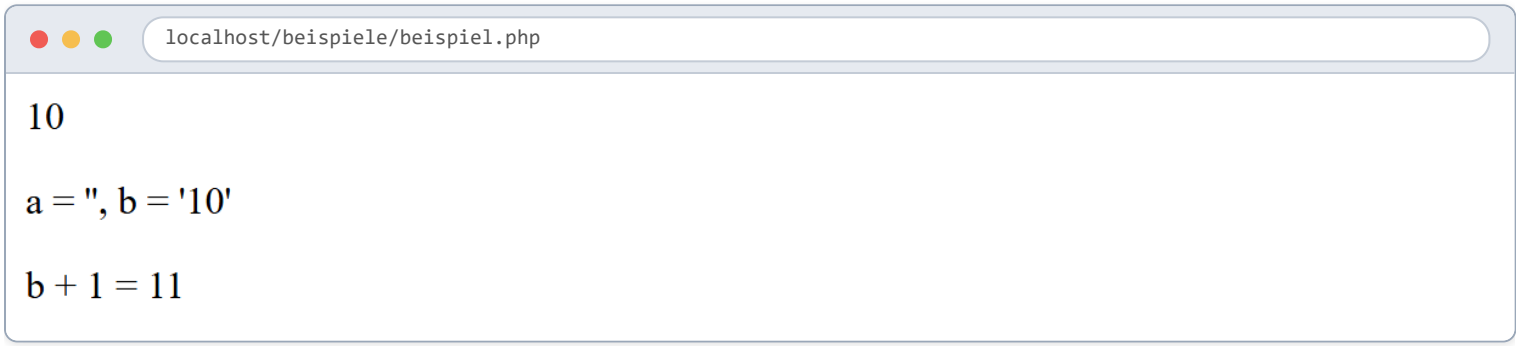


Abbildung 3: mit_echo() gibt 10 aus, liefert aber nichts zurück.

> **FAUSTREGEL**
>
> Funktionen, die etwas **berechnen**, geben das Ergebnis mit `return` zurück und geben selbst nichts aus. So bleiben sie überall verwendbar – in einem Satz, in einer Tabelle, in einer weiteren Rechnung.

3 | Mehrere Parameter und Standardwerte

Parameter werden durch Kommas getrennt und in derselben Reihenfolge übergeben. Bekommt ein Parameter mit `=` einen **Standardwert**, darf man ihn beim Aufruf weglassen.

Parameter mit Standardwert beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Parameter</title></head>
<body>

<?php
  function becherpreis($kugeln, $kugelpreis, $aufpreis = 2.20) {
    return $kugeln * $kugelpreis + $aufpreis;
  }

  echo "<p>" . becherpreis(3, 1.70) . "</p>";         // Aufpreis: Standardwert
  echo "<p>" . becherpreis(3, 1.70, 3.50) . "</p>";   // Aufpreis: 3,50
?>

</body>
</html>
```

SO SIEHT ES IM BROWSER AUS



Abbildung 4: Beim ersten Aufruf gilt der Standardwert 2,20 €.

4 | Der Gültigkeitsbereich

Eine Funktion ist ein abgeschlossener Raum: Variablen von außen sieht sie **nicht**, und ihre eigenen Variablen verschwinden nach dem `return`. Was sie braucht, bekommt sie über Parameter.

Fehler: Zugriff auf eine äußere Variable beispiel.php

```php
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Gültigkeitsbereich</title></head>
<body>

<?php
  $kugelpreis = 1.70;

  function preis($anzahl) {
    return $anzahl * $kugelpreis;     // $kugelpreis ist hier unbekannt!
  }

  echo "<p>3 Kugeln: " . preis(3) . "</p>";
?>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS

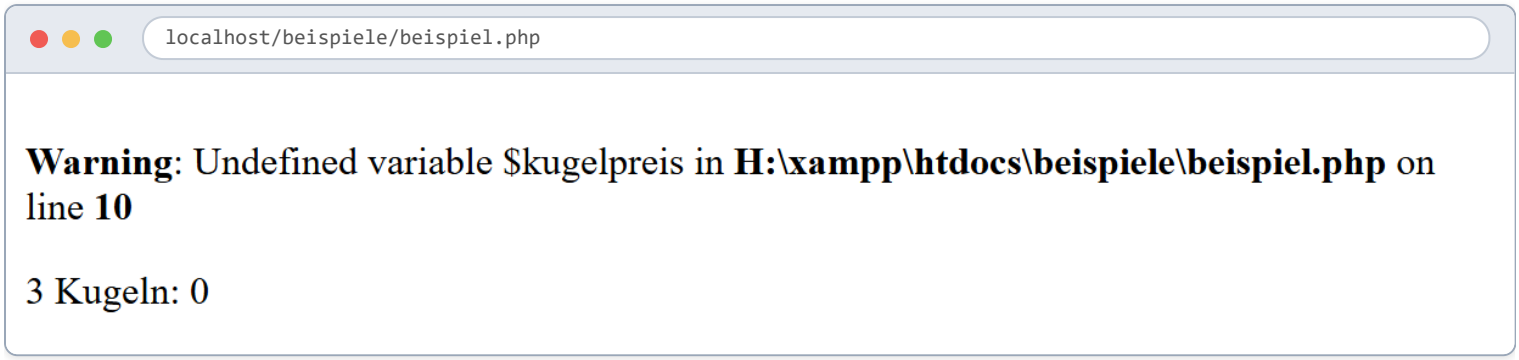


Abbildung 5: Warnung: In der Funktion gibt es keine Variable $kugelpreis.

Richtig: Wert als Parameter übergeben

```
function preis($anzahl, $kugelpreis) {
  return $anzahl * $kugelpreis;
}
echo preis(3, $kugelpreis);
```

> 💡 **WARUM SO STRENG?**
>
> Weil eine Funktion so unabhängig bleibt: Sie funktioniert in jeder Datei und mit jedem Wert, ohne dass man wissen muss, welche Variablen es gerade draußen gibt. Das Schlüsselwort `global` hebt die Trennung auf – es gilt als schlechter Stil und wird in dieser Reihe nicht verwendet.

5 | Eingebaute Funktionen

PHP bringt über tausend Funktionen mit. Die Dokumentation auf `php.net/manual/de` beschreibt jede mit Parametern und Beispielen. Einige nützliche:

Eine kleine Auswahl beispiel.php

```
<!DOCTYPE html>
<html lang="de">
<head><meta charset="UTF-8"><title>Eingebaute Funktionen</title></head>
<body>

<?php
  $sorte = "pistazie";
  echo "<p>" . ucfirst($sorte) . " hat " . strlen($sorte) . " Buchstaben.</p>";
  echo "<p>" . strtoupper("neu") . " – " . str_replace("Eis", "Gelato", "Eiscafé") . "</p>";
  echo "<p>round(4.567, 1) = " . round(4.567, 1) . ", ceil(4.1) = " . ceil(4.1)
       . ", floor(4.9) = " . floor(4.9) . "</p>";
  echo "<p>Größter Wert: " . max(3, 9, 4) . ", Zufallszahl 1–6: " . random_int(1, 6) . "</p>";
?>

</body>
</html>
```

👁 SO SIEHT ES IM BROWSER AUS

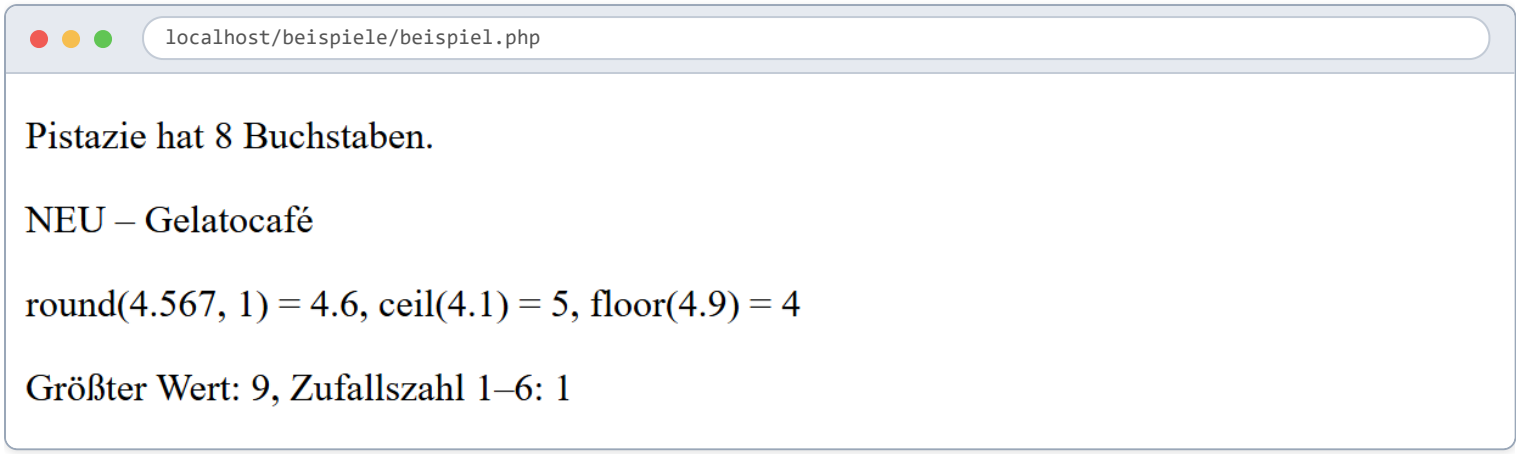


Abbildung 6: Texte umformen, runden, Maximum und Zufall.

Funktion	Aufgabe
`strlen($t)`	Länge eines Textes (Umlaute zählen doppelt, dafür gibt es `mb_strlen`)
`strtoupper($t)` / `strtolower($t)`	Groß- bzw. Kleinschreibung
`ucfirst($t)`	erster Buchstabe groß
`str_replace($alt, $neu, $t)`	Textteile ersetzen
`round($z, $st)`, `ceil($z)`, `floor($z)`	runden, aufrunden, abrunden
`max(…)`, `min(…)`	größter / kleinster Wert
`random_int($von, $bis)`	Zufallszahl